ΠΑΝΕΛΛΑΔΙΚΕΣ ΕΞΕΤΑΣΕΙΣ

ΗΜΕΡΗΣΙΩΝ – ΕΣΠΕΡΙΝΩΝ ΕΠΑΓΓΕΛΜΑΤΙΚΩΝ ΛΥΚΕΙΩΝ

ΤΡΙΤΗ 11 ΙΟΥΝΙΟΥ 2024

ΕΞΕΤΑΖΟΜΕΝΟ ΜΑΘΗΜΑ: ΠΡΟΓΡΑΜΜΑΤΙΣΜΟΣ ΥΠΟΛΟΓΙΣΤΩΝ

ΘΕΜΑ Α

Α1. Να χαρακτηρίσετε τις προτάσεις που ακολουθούν, γράφοντας στο τετράδιό σας, δίπλα στο γράμμα που αντιστοιχεί σε κάθε πρόταση, τη λέξη Σωστό, αν η πρόταση είναι σωστή, ή τη λέξη Λάθος, αν η πρόταση είναι λανθασμένη.

α. Η σύνταξη για την κλήση μιας μεθόδου είναι διαφορετική από τη σύνταξη για την κλήση μίας συνάρτησης.

β. Στην Python η δομή της ουράς μπορεί να υλοποιηθεί με μία λίστα στην οποία οι εισαγωγές και οι εξαγωγές στοιχείων γίνονται μόνο από το ένα άκρο.

γ. Στην Python δεν μπορούμε να εκχωρήσουμε διαφορετικούς τύπους τιμών σε μία μεταβλητή κατά τη διάρκεια ενός προγράμματος.

δ. Η τιμή της λογικής έκφρασης x and not x είναι πάντα True ανεξάρτητα από την τιμή της λογικής μεταβλητής x.

ε. Οι τοπικές μεταβλητές μπορούν να προσπελαστούν μόνο μέσα στη συνάρτηση στην οποία δηλώθηκαν.

α. ΣΩΣΤΟ **β.** ΛΑΘΟΣ **γ.** ΛΑΘΟΣ **δ.** ΛΑΘΟΣ **ε.** ΣΩΣΤΟ

Α2. Να γράψετε στο τετράδιό σας τους αριθμούς 1, 2, 3, 4, 5 από τη στήλη Α του παρακάτω πίνακα και δίπλα σε καθέναν από τους αριθμούς, ένα από τα γράμματα α, β, γ, δ, ε, στ της στήλης Β, που δίνει τη σωστή αντιστοίχιση. Σημειώνεται ότι ένα γράμμα από τη στήλη Β θα περισσέψει.

ΣΤΗΛΗ Α	ΣΤΗΛΗ Β
1. int("5")**2	α. (5,0)
2. str(5)*2	β. 25
3. "5500"[:1]	γ. 1
4. pow(5,0)	δ. "5"
5. divmod(25,5)	ε. (0,5)
	στ. "55"

1. β **2**. στ **3**. δ **4**. γ **5**. α

ΘΕΜΑ Β

Β1. Να γράψετε συνάρτηση trim_a που να δέχεται μία συμβολοσειρά s1 και να επιστρέφει μία νέα συμβολοσειρά, όμοια με την s1, χωρίς τους χαρακτήρες ΄a΄ και ΄Α΄. Για παράδειγμα, αν κληθεί με s1 = "Alabama" πρέπει να επιστρέψει τη συμβολοσειρά "lbm".

```
def trim_a(s1):
    s2 = '' # Αρχικοποιούμε μια κενή συμβολοσειρά που θα "χτίσουμε" σταδιακά

    for char in s1: # Διατρέχουμε έναν-έναν όλους τους χαρακτήρες της αρχικής συμβολοσειράς s1

        if char != 'a' and char != 'A': # Ελέγχουμε αν ο τρέχων χαρακτήρας ΔΕΝ είναι 'a' ΚΑΙ ΔΕΝ είναι 'A'

            s2 = s2 + char # Αν η συνθήκη ισχύει, τον προσθέτουμε στη νέα μας συμβολοσειρά s2
    return s2 # Αφού τελειώσει η επανάληψη, επιστρέφουμε το τελικό αποτέλεσμα
```

Προσοχή στην Python το 'a' και το 'A' είναι δύο τελείως διαφορετικοί χαρακτήρες (διαφορετικοί κωδικοί ASCII/Unicode).

Β2. Δίνεται η παρακάτω λίστα:

0	1	2	3	4	5	6	7	8	9
235	181	145	98	73	42	31	29	12	7

Να εκτελέσετε τον αλγόριθμο της δυαδικής αναζήτησης για:
α) τον αριθμό 85
β) τον αριθμό 12
Να γράψετε στο τετράδιό σας τους αριθμούς που θα συγκριθούν κατά την εκτέλεση του αλγορίθμου σε κάθε μία από τις παραπάνω περιπτώσεις.

α) Χρησιμοποιούμε τις μεταβλητές αρχή, τέλος και μέσο (mid = (αρχή + τέλος) / 2)

1η Επανάληψη:

- αρχή = 0, τέλος = 9
- μέσο = (0+9) / 2 = 4
- Σύγκριση με την τιμή **73**.
- Επειδή 85 > 73 (και ο πίνακας είναι φθίνων), το 85 πρέπει να βρίσκεται "αριστερά" (σε μικρότερους δείκτες). Άρα: τέλος = μέσο - 1 = 3

2η Επανάληψη:

- αρχή = 0, τέλος = 3
- μέσο = (0+3) / 2 = 1
- Σύγκριση με την τιμή **181**.
- Επειδή 181 > 85, το 85 πρέπει να βρίσκεται "δεξιά". Άρα: αρχή = μέσο + 1 = 2.

3η Επανάληψη:

- αρχή = 2, τέλος = 3
- μέσο = (2+3) / 2 = 2
- Σύγκριση με την τιμή **145**.
- Επειδή 145>85, το 85 πρέπει να βρίσκεται "δεξιά". Άρα: αρχή = μέσο + 1 = 3.

4η Επανάληψη:

- αρχή = 3, τέλος = 3
- μέσο = (3+3) / 2 = 3
- Σύγκριση με την τιμή **98**.
- Επειδή 98> 85, το 85 πρέπει να βρίσκεται "δεξιά". Άρα: αρχή = μέσο + 1 = 4.

85: Συγκρίνονται οι αριθμοί **73**, **181**, **145**, **98**.

β) 1η Επανάληψη:

- αρχή = 0, τέλος = 9
- μέσο = (0+9) / 2 = 4
- Σύγκριση με την τιμή **73**.
- Επειδή 73 >12, το 12 βρίσκεται "δεξιά" (μεγαλύτεροι δείκτες). Άρα: αρχή = μέσο + 1 = 5.

2η Επανάληψη:

- αρχή = 5, τέλος = 9
- μέσο = (5+9) // 2 = 7
- Σύγκριση με την τιμή **29**.
- Επειδή 29 > 12, το 12 βρίσκεται "δεξιά". Άρα: αρχή = μέσο + 1 = 8.

3η Επανάληψη:

- αρχή = 8, τέλος = 9
- μέσο = (8+9) / 2 = 8
- Σύγκριση με την τιμή **12**.
- Επειδή 12 = 12, ο αριθμός **βρέθηκε**.

12: Συγκρίνονται οι αριθμοί **73**, **29**, **12**.

Β3. Δίνεται το παρακάτω τμήμα προγράμματος σε γλώσσα προγραμματισμού Python:

```
for i in range (10):
    for  j  in range (10, -1, 1):
        print i*j
```

Να ξαναγράψετε στο τετράδιό σας το παραπάνω τμήμα προγράμματος, αντικαθιστώντας τις εντολές **for** με **while**, έτσι ώστε να εμφανίζεται το ίδιο αποτέλεσμα.

```
i = 0                    # Αρχικοποίηση του εξωτερικού μετρητή
while i < 10:            # Εξωτερική επανάληψη (0 έως 9)
    j = 10               # Αρχικοποίηση του εσωτερικού μετρητή
    while j < -1:        # ΕΛΕΓΧΟΣ: Εδώ λες "όσο το 10 είναι μικρότερο του -1" (Πάντα
Ψευδές!)
        print(i * j)     # Αυτή η γραμμή δεν θα εκτελεστεί ποτέ (όπως και στην αρχική
for)
        j = j + 1        # ΒΗΜΑ: Αύξηση του j (όπως το 1 στην range)
    i = i + 1            # ΒΗΜΑ: Αύξηση του i
```

ΘΕΜΑ Γ

Σε έναν διαγωνισμό μαθηματικών συμμετέχουν μαθητές από όλη τη χώρα. Κάθε διαγωνιζόμενος βαθμολογείται σε δέκα (10) θέματα. Η βαθμολογία κάθε θέματος είναι από 1 έως και 20 μονάδες. Η τελική βαθμολογία κάθε διαγωνιζόμενου προκύπτει από τον μέσο όρο των βαθμολογιών των δέκα (10) θεμάτων. Στην επόμενη φάση του διαγωνισμού προκρίνονται οι διαγωνιζόμενοι που έχουν τελική βαθμολογία μεγαλύτερη από 15 και σε όλα τα θέματα έχουν βαθμό μεγαλύτερο ή ίσο του 12.

Να αναπτύξετε πρόγραμμα σε γλώσσα προγραμματισμού Python το οποίο:

Γ1. Για κάθε διαγωνιζόμενο:
α) Να διαβάζει το ονοματεπώνυμό του .
β) Να διαβάζει τη βαθμολογία για καθένα εκ των δέκα (10) θεμάτων με έλεγχο ορθότητας τιμών.
γ) Η εισαγωγή των δεδομένων να τερματίζει, όταν δοθεί ως ονοματεπώνυμο διαγωνιζόμενου η λέξη "ΤΕΛΟΣ". Θεωρήστε ότι υπάρχουν τουλάχιστον δύο (2) διαγωνιζόμενοι.

Γ2. Να υπολογίζει και να εμφανίζει την τελική βαθμολογία κάθε διαγωνιζόμενου.

Γ3. Να εμφανίζει κατάλληλο μήνυμα, εάν ο διαγωνιζόμενος προκρίνεται στην επόμενη φάση.

Γ4. Να βρίσκει και να εμφανίζει τη μεγαλύτερη τελική βαθμολογία.

Γ5. Να υπολογίζει και να εμφανίζει το ποσοστό των διαγωνιζόμενων που προκρίθηκαν στην επόμενη φάση.

```
# ΘΕΜΑ Γ
megistos = 0  # Γ4: Αρχικοποίηση μέγιστης τελικής βαθμολογίας (αντί για max)
p = 0  # Αρχικοποίηση πλήθους όλων των μαθητών στον διαγωνισμό
p2 = 0   # ΠΡΟΣΟΧΗ: Αρχικοποίηση πλήθους μαθητών που προκρίθηκαν (Αυτό έλειπε!)

onoma = raw_input()   # Γ1.α: Διάβασμα ονόματος 1ου διαγωνιζόμενου
while onoma != 'ΤΕΛΟΣ':   # Γ1.γ: Έλεγχος τερματισμού επαναληπτικής διαδικασίας
    p = p + 1  # Αυξάνουμε τον συνολικό μετρητή μαθητών κατά 1
    s = 0     # Αρχικοποίηση αθροίσματος βαθμών για τον τρέχοντα μαθητή
    p1 = 0    # Αρχικοποίηση μετρητή: πλήθος μαθημάτων με βαθμό >= 12

    for i in range(10):  # Γ1.β: Επανάληψη για τα 10 θέματα του μαθητή
        vath = int(input())              # Διάβασμα του βαθμού
        while vath < 1 or vath > 20:     # Γ1.β: Αμυντικός προγραμματισμός - Έλεγχος
ορθότητας τιμών
            vath = int(input())  # Ξαναδιαβάζει αν δοθεί λάθος βαθμός (εκτός 1-20)

        s = s + vath     # Προσθήκη του έγκυρου βαθμού στο άθροισμα (s)
        if vath >= 12:  # Έλεγχος αν ο συγκεκριμένος βαθμός είναι μεγαλύτερος ή ίσος
του 12
            p1 = p1 + 1 # Αν ναι, αυξάνουμε τον μετρητή των επιτυχημένων μαθημάτων (p1)

    mo = s / 10.0 # Γ2: Υπολογισμός μέσου όρου (Το 10.0 εξασφαλίζει δεκαδικό αποτέλεσμα
στην Python 2)
    print mo  # Γ2: Εμφάνιση της τελικής βαθμολογίας (Μ.Ο.)

    # Γ3: Συνθήκη πρόκρισης: Μ.Ο. > 15 ΚΑΙ ο μαθητής να έχει γράψει >=12 ΚΑΙ στα 10
μαθήματα (άρα p1 == 10)
    if mo > 15 and p1 == 10:
        print 'ΠΡΟΚΡΙΝΕΤΑΙ ΣΤΗΝ ΕΠΟΜΕΝΗ ΦΑΣΗ' # Γ3: Εμφάνιση μηνύματος πρόκρισης
        p2 = p2 + 1       # Αυξάνουμε τον μετρητή των μαθητών που προκρίθηκαν

    if mo > megistos:       # Γ4: Εύρεση της μεγαλύτερης τελικής βαθμολογίας
        megistos = mo       # Ενημέρωση του μέγιστου βαθμού

    onoma = raw_input()     # Γ1.α: Διάβασμα ονόματος επόμενου διαγωνιζόμενου

pososto = float(p2) / p * 100  # Γ5: Υπολογισμός ποσοστού (Η χρήση float(p2)
print megistos, pososto  # Γ4 & Γ5: Εμφάνιση μέγιστου και ποσοστού

# Δεν χρειάζεται if p > 0 για την αποφυγή διαίρεσης με το μηδέν, γιατί η εκφώνηση
εξασφαλίζει τουλάχιστον 2 διαγωνιζόμενους.
```

ΘΕΜΑ Δ

Δίνεται το αρχείο "branch.txt", το οποίο περιέχει ονόματα υποκαταστημάτων μίας εταιρείας. Το αρχείο έχει ενδεικτικά την παρακάτω δομή:

Micro_Vironas

Mega_Kifisia

.

.

.

Σε κάθε γραμμή του αρχείου "branch.txt" βρίσκεται το όνομα του υποκαταστήματος.
Να αναπτύξετε πρόγραμμα σε γλώσσα προγραμματισμού Python το οποίο:

Δ1. Να διαβάζει το αρχείο "branch.txt" γραμμή προς γραμμή και να καταχωρίζει τα ονόματα των υποκαταστημάτων σε μία λίστα με όνομα ON. Θεωρήστε ότι στο αρχείο υπάρχουν τουλάχιστον δύο (2) υποκαταστήματα.

Δ2. Για κάθε υποκατάστημα:
α) Να διαβάζει την ημερήσια είσπραξη για κάθε μία από τις τριάντα (30) ημέρες του μήνα Ιουνίου.
β) Να υπολογίζει το σύνολο των εισπράξεων του μήνα Ιουνίου και να το καταχωρίζει σε λίστα με όνομα S_POSO.

Δ3. α) Να υπολογίζει και να εμφανίζει τον μέσο όρο των εισπράξεων όλων των υποκαταστημάτων για τον μήνα Ιούνιο.
β) Να υπολογίζει και να εμφανίζει το πλήθος των υποκαταστημάτων που έχουν εισπράξεις μεγαλύτερες ή ίσες του μέσου όρου.

Δ4. Να ταξινομεί τη λίστα S_POSO με χρήση του αλγορίθμου ταξινόμησης της ευθείας ανταλλαγής (φυσαλίδα – bubble sort) σε φθίνουσα σειρά, αναδιατάσσοντας συγχρόνως τη λίστα ΟΝ, έτσι ώστε να διατηρείται η αντιστοιχία ονομάτων υποκαταστημάτων – εισπράξεων.
Σε περίπτωση ισότητας εισπράξεων να γίνεται ταξινόμηση με αλφαβητική σειρά ως προς τα ονόματα της λίστας ΟΝ.

```
# ΘΕΜΑ Δ
ON = []  # Δ1: Δημιουργία κενής λίστας για τα ονόματα των υποκαταστημάτων

f = open('branch.txt', 'r') # Άνοιγμα του αρχείου για ανάγνωση

for line in f:      # Διάβασμα του αρχείου γραμμή προς γραμμή
    ON.append(line)  # ΕΝΤΟΣ ΥΛΗΣ: Εισαγωγή της γραμμής αυτούσιας, όπως δείχνει το
βιβλίο

f.close()           # Κλείσιμο του αρχείου

S_POSO = []  # Δ2β: Κενή λίστα για τις συνολικές εισπράξεις
for i in range(len(ON)):  # Δ2: Επανάληψη για κάθε υποκατάστημα
    s = 0.0       # Αρχικοποίηση αθροίσματος (χρήση float εξ αρχής για ποσά)
    for j in range(30):      # Δ2α: Επανάληψη για τις 30 ημέρες
        eisp = input("Δώσε είσπραξη ημέρας: ")  # Το input() στο βιβλίο αρκεί για
αριθμούς
        s = s + eisp   # Προσθήκη της είσπραξης στο συνολικό άθροισμα του
    S_POSO.append(s)  # Δ2β: Καταχώριση του μηνιαίου συνόλου

s1 = 0.0 # Δ3α: Γενικό σύνολο εταιρείας
for x in S_POSO: # Διάσχιση της λίστας των εισπράξεων (πολύ σωστή επιλογή αντί για
range)
    s1 = s1 + x  # Προσθήκη κάθε ποσού στο γενικό σύνολο

mo = s1 / len(S_POSO)  # Υπολογισμός μέσου όρου
print "Μέσος Όρος Εισπράξεων: ", mo  # Εμφάνιση του μέσου όρου

p = 0 # Δ3β: Αρχικοποίηση μετρητή για υποκαταστήματα με εισπράξεις >= Μ.Ο.
for x in S_POSO: # Νέα διάσχιση της λίστας των εισπράξεων
    if x >= mo: # Έλεγχος αν η είσπραξη ικανοποιεί το κριτήριο
        p = p + 1  # Αν ναι, αυξάνεται ο μετρητής
print "Πλήθος υποκαταστημάτων >= Μ.Ο.: ", p # Εμφάνιση του πλήθους των υποκαταστημάτων

N = len(S_POSO)  # Δ4: Πλήθος στοιχείων για τη Φυσαλίδα
for i in range(N - 1):  # Αλγόριθμος Φυσαλίδας - Εξωτερικός βρόχος (περάσματα)
    for j in range(N - 1, i, -1): # Εσωτερικός βρόχος - Ξεκινάει από το τέλος προς την
αρχή

        # Κριτήριο 1: Φθίνουσα ως προς τις εισπράξεις (μεγαλύτερο -> πάνω)
        if S_POSO[j] > S_POSO[j-1]:
            S_POSO[j], S_POSO[j-1] = S_POSO[j-1], S_POSO[j] # Αντιμετάθεση στη λίστα
εισπράξεων
            ON[j], ON[j-1] = ON[j-1], ON[j] # Παράλληλη μετακίνηση ονομάτων

        # Κριτήριο 2: Ισοβαθμία εισπράξεων -> Αλφαβητική (Αύξουσα) στα ονόματα
```

```
        elif S_POSO[j] == S_POSO[j-1]:
            if ON[j] < ON[j-1]: # Ταξινόμηση ΑΛΦΑΒΗΤΙΚΗ (Αύξουσα) ως προς τα ονόματα
                # Αντιμετάθεση μόνο στα ονόματα (τα ποσά S_POSO είναι ίσα, οπότε δεν χρειάζεται να αλλάξουν)
                ON[j], ON[j-1] = ON[j-1], ON[j]
```

1. **Παράλληλες Λίστες:** Είναι το πιο κλασικό θέμα εξετάσεων. Πρέπει να θυμάστε ότι όταν το S_POSO κάνει "swap", το ON **πρέπει πάντα** να ακολουθεί (αντιμετάθεση), αλλιώς τα ποσά θα αντιστοιχούν σε λάθος υποκαταστήματα.
2. **Διπλό Κριτήριο (Bubble Sort):** Η λογική του elif. Πρώτα κοιτάμε το ποσό είναι η λίστα **S_POSO** . *Μόνο αν τα ποσά είναι ίδια* κοιτάμε το αλφαβητικό κριτήριο που είναι η λίστα **ON** .